

**Exercise 2.7:** Consider the problem,

$$\text{minimize } f(x_1, x_2) = (x_2 - x_1^2)(x_2 - 2x_1^2)$$

- (i) Show that the first- and second order necessary conditions for optimality are satisfied at  $(0, 0)^T$ .

*Solution.* First we compute  $\nabla f(x_1, x_2)$  and  $\nabla^2 f(x_1, x_2)$

$$\nabla f(x_1, x_2) = (8x_1^3 - 6x_2x_1, 2x_2 - 3x_1^2)^T$$

$$\nabla^2 f(x_1, x_2) = \begin{pmatrix} 24x_1^2 - 6x_2 & -6x_1 \\ -6x_1 & 2 \end{pmatrix}$$

Checking the first order necessary condition,  $\nabla f(0, 0) = (8(0)^3 - 6(0)(0), 2(0) - 3(0)^2) = (0, 0)^T$ . Checking the second order necessary condition. Computing the determinant we get  $\det(\nabla^2 f(0, 0)) = 0$ . This Hessian is not definite therefore second order necessary conditions are not satisfied at  $(0, 0)^T$ .  $\square$

- (ii) Show that the origin is a local minimizer of  $f$  along any line passing through the origin (that is,  $x_2 = mx_1$ ).

*Solution.* Let  $x_2 = mx_1$  and by substitution into  $f(x_1, x_2)$  we get  $f(x_1) = (mx_1 - x_1^2)(mx_1 - 2x_1^2) = m^2x_1^2 - 3mx_1^3 + 2x_1^4$ . Computing the first and second derivatives we get  $f'(x_1) = 2m^2x_1 - 9mx_1^2 + 8x_1^3$ ,  $f''(x_1) = 2m^2 - 18mx_1 + 24x_1^2$  and clearly  $x_1 = 0$  produces a minimum with  $f'(0) = 0$  and  $f''(0) = 2m^2$ .  $\square$

- (iii) Show that the origin is not a local minimizer of  $f$  (consider, for example, curves of the form  $x_2 = kx_1^2$ ). What conclusions can you draw from this?

*Solution.* Let  $x_2 = kx_1^2$  and by substitution into  $f(x_1, x_2)$  we get  $f(x_1) = (kx_1^2 - x_1^2)(kx_1^2 - 2x_1^2) = k^2x_1^4 - 3kx_1^4 + 2x_1^4 = (k^2 - 3k + 2)x_1^4$ . Computing the first derivative we get  $f'(x_1) = 4(k^2 - 3k + 2)x_1^3$  and plugging in  $x_1 = 0$  we see that  $f'(0) = 0$ . Applying the first derivative test, suppose  $k$  is chosen such that  $(k^2 - 3k + 2) > 0$  and let  $\epsilon > 0$ . Note that  $f'(-\epsilon) = 4(k^2 - 3k + 2)(-\epsilon)^3 = -4(k^2 - 3k + 2)(\epsilon)^3 < 0$  and also note that  $f'(\epsilon) = 4(k^2 - 3k + 2)(\epsilon)^3 > 0$ . Thus when  $k$  is chosen such that  $(k^2 - 3k + 2) > 0$ ,  $f(0, 0)$  becomes a local maximum along the curve  $x_2 = kx_1^2$ .

We can conclude that  $f(x_1, x_2)$  is a saddle.  $\square$

**Exercise 2.10:** Let

$$f(x) = cx_1^2 + x_2^2 - 2x_1x_2 - 2x_2$$

where  $c$  is some scalar.

- (i) Determine the stationary points of  $f$  for each value of  $c$ .

*Solution.* The condition for a stationary point is

$$\nabla f(x) = \begin{pmatrix} 2cx_1 - 2x_2 \\ 2x_2 - 2x_1 - 2 \end{pmatrix} = 0.$$

We conclude that  $x_1 = 1/(c - 1)$  and  $x_2 = c/(c - 1)$ . □

- (ii) For what values of  $c$  can  $f$  have a minimizer? For what values of  $c$  can  $f$  have a maximizer? Determine the minimizers/maximizers corresponding to such values of  $c$  and indicate what kind of minima or maxima they are.

*Solution.* The condition for  $x$  being a minimizer is  $\nabla^2 f(x)$  being positive definite. Computing the Hessian,

$$\nabla^2 f(x) = \begin{pmatrix} 2c & -2 \\ -2 & 2 \end{pmatrix}$$

Since  $\nabla^2 f(x)$  is  $2 \times 2$  the sign of the determinant gives us definiteness. Note that  $\det(\nabla^2 f(x)) = 4c - 4 = 4(c - 1)$ . When  $c > 1$  the determinant is positive and we have definiteness. Note that performing the row operation  $r_2 = r_2 + \frac{r_1}{c}$  gives,

$$\nabla^2 f(x) = \begin{pmatrix} 2c & -2 \\ 0 & 2 - \frac{2}{c} \end{pmatrix}.$$

Recall that since  $\nabla^2 f(x)$  is symmetric it is also positive definite if all its pivots are positive. Note that when  $c > 1$  we get  $2c, 2 - \frac{2}{c} > 0$ . Thus this matrix is only ever positive definite, specifically when  $c > 1$ .

So when  $c > 1$ ,  $f$  has a minimizer,  $f$  will never have a maximizer for any value of  $c$ . The minimizer when  $c > 1$  is  $(1/(c - 1), c/(c - 1))$  and this must be a global minimizer since  $\nabla^2 f(x)$  is constant for all values of  $x$ . □

**Exercise 3.2:** Use Newton's method to solve

$$\text{minimize } f(x) = 5x^5 + 2x^3 - 4x^2 - 3x + 2$$

Look for a solution in the interval  $-2 \leq x \leq 2$ . Make sure that you have found a minimum and not a maximum. You may want to experiment with different initial guesses of the solution.

*Solution.* Newton's method for optimization requires us to find  $f'(x)$  and  $f''(x)$ ,

$$f'(x) = 25x^4 + 6x^2 - 8x - 3,$$

$$f''(x) = 100x^3 + 12x - 8.$$

With our iteration being,

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

Consider the following code

**Console:**

```
>> fp = @(x) 25*x.^4 + 6*x.^2 - 8*x - 3
      @(x)25*x.^4+6*x.^2-8*x-3
>> fpp = @(x) 100*x.^3 + 12*x - 8
      @(x)100*x.^3+12*x-8

-----
>> x = .5
      0.5000000000000000
>> x = x - fp(x)/fpp(x)
      0.8750000000000000
>> x = x - fp(x)/fpp(x)
      0.741916104553120
>> x = x - fp(x)/fpp(x)
      0.695393499199703
>> x = x - fp(x)/fpp(x)
      0.689967427734573
>> x = x - fp(x)/fpp(x)
      0.689897959836467
>> x = x - fp(x)/fpp(x)
      0.689897948556636
>> x = x - fp(x)/fpp(x)
      0.689897948556636

-----
>> fp(x)
      1.776356839400250e-15
>> fpp(x)
      33.115101530718512
```

```
>> fx = 5*x.^5 + 2*x.^3 - 4*x.^2 - 3*x + 2
      -0.535366497965985
```

```
>> x = -2
      -2
```

```
>> fx = 5*x.^5 + 2*x.^3 - 4*x.^2 - 3*x + 2
      -184
```

Note that  $f''(x) > 0$  therefore our solution is a minimum. Clearly our solution is a local minimum even on the interval  $-2 \leq x \leq 2$  since  $f(-2) < f(x)$ .  $\square$

**Exercise 3.3:** Use Newton's method to solve,

$$\text{minimize } f(x_1, x_2) = 5x_1^4 + 6x_2^4 - 6x_1^2 + 2x_1x_2 + 5x_2^2 + 15x_1 - 7x_2 + 12.$$

Use the initial guess  $(1, 1)^T$ . Make sure that you have found the minimum and not the maximum.

*Solution.* Newton's method for optimization requires us to find  $\nabla f(x)$  and  $\nabla^2 f(x)$ ,

$$\nabla f(x) = \begin{pmatrix} 20x_1^3 - 12x_1 + 2x_2 + 15 \\ 24x_2^3 + 2x_1 + 10x_2 - 7 \end{pmatrix}$$

$$\nabla^2 f(x) = \begin{pmatrix} 60x_1^2 - 12 & 2 \\ 2 & 72x_2^2 + 10 \end{pmatrix}$$

With our iteration being,

$$x_{n+1} = x_n - [\nabla^2 f(x_n)]^{-1} \nabla f(x_n).$$

Really we are going to be using,

$$[\nabla^2 f(x_n)]^{-1} p = -\nabla f(x_n)$$

where  $x_{n+1} = x_n + p$ . Consider the following code

**Console:**

```
>> gf = @(x, y) [ 20*x.^3 - 12*x + 2*y + 15; 24*y.^3 + 2*x + 10*y - 7]
>> H = @(x, y) [60*x.^2 - 12, 2; 2, 72*y.^2 + 10]
>> x = [1; 1]

-----
>> x = x + (H(x(1), x(2)) \ (-1.*gf(x(1), x(2))))
      0.493387589013225
      0.658697863682604

>> x = x + (H(x(1), x(2)) \ (-1.*gf(x(1), x(2))))
      -4.464349611822802
```

0.718896731093291

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-3.020177682334610
0.653965250260218
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-2.088864692018031
0.603115349257777
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.524073628823325
0.568625690678257
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.238180450432060
0.549922144035557
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.150429606728832
0.543952868733066
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.142126719774461
0.543377489261745
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.142054933710197
0.543372481578204
```

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.142054928369237
0.543372481205087
```

Repeat

```
>> x = x + (H(x(1), x(2))\(-1.*gf(x(1), x(2))))
-1.142054928369237
0.543372481205087
```

-----

```
>> eig(H(x(1), x(2)))
31.144345190282863
66.371285414294931
```

```
>> norm(gf(x(1), x(2)))
5.329070518200751e-15
```

Note that  $\nabla^2 f(x)$  is positive definite and  $\nabla f(x) = 0$  therefore our solution is a minimum.  $\square$

**Exercise 3.4:** Consider the problem

$$\text{minimize } f(x) = x^4 - 1.$$

Solve this problem using Newton's method. Start from  $x_0 = 4$  and perform three iterations. Prove that the iterates converge to the solution. What is the rate of convergence? Can you explain this?

*Solution.* To do Newton's method for optimization we must first note that  $f'(x) = 4x^3$  and  $f''(x) = 12x^2$ . Now consider the Newton iteration,

$$x_{n+1} = x_n - \frac{4x_n^3}{12x_n^2} = x_n - \frac{1}{3}x_n = \frac{2}{3}x_n$$

With  $x_0 = 4$  we get that

$$x_n = \left(\frac{2}{3}\right)^n (4).$$

The first three iterations are  $x_1 \approx 2.666$ ,  $x_2 \approx 1.777$ , and  $x_3 \approx 1.185$ . Note that the limit,

$$\lim_{n \rightarrow \infty} 4 \left(\frac{2}{3}\right)^n = 4 \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0.$$

Therefore this sequence must converge to 0 which is the solution to the minimization problem. Clearly the rate of convergence is  $2/3$  (linear convergence) since  $x_{n+1}/x_n = 2/3$ . Each new term in the sequence is two thirds the size of the previous one, hence the rate of convergence is  $2/3$ .  $\square$

**Exercise 3.6:** Consider the problem

$$\text{minimize } f(x) = \frac{1}{2}x^T Qx - c^T x$$

where  $Q$  is a positive-definite matrix. Prove that Newton's method will determine the minimizer of  $f$  in one iteration, regardless of the starting point.

*Solution.* Performing Newton's method on this problem we first must compute the  $\nabla f(x)$  and  $\nabla^2 f(x)$ . Note that,

$$\nabla f(x) = Qx - c,$$

$$\nabla^2 f(x) = Q.$$

Computing and simplifying the Newton step we get

$$\begin{aligned} x_{n+1} &= x_n - [\nabla^2 f(x_n)]^{-1} \nabla f(x_n), \\ &= x_n - Q^{-1}(Qx_n - c), \\ &= x_n - Q^{-1}Qx_n + Q^{-1}c, \\ &= x_n - x_n + Q^{-1}c, \\ &= Q^{-1}c. \end{aligned}$$

So Newton's method on this problem will converge at  $Q^{-1}c$  after the first iteration. Note that

$$\nabla f(Q^{-1}c) = Q(Q^{-1}c) - c = 0$$

and since  $Q$  is positive-definite,  $Q^{-1}c$  is a minimizer for the problem.  $\square$

**Exercise P13:** Let  $c \in \mathbb{R}^n$  and suppose  $Q \in \mathbb{R}^{n \times n}$  is a symmetric positive definite matrix. Consider this quadratic function on  $x \in \mathbb{R}^n$

$$f(x) = \frac{1}{2}x^T Qx - c^T x$$

(a) Show that  $\nabla f(x) = Qx - c$ . (This is shown explicitly in Appendix B5)

*Proof.* Solution Note that we can rewrite the function in terms of sums,

$$f(x) = \frac{1}{2} \sum_{j=1}^n \sum_{k=1}^n Q_{j,k} x_j x_k - \sum_{j=1}^n b_j x_j.$$

For every  $x_i \in x$  we have the following,

$$\frac{1}{2} Q_{i,i} x_i^2 + \left( \frac{1}{2} \sum_{j \neq i} Q_{j,i} x_j x_i \right) + \left( \frac{1}{2} \sum_{k \neq i} Q_{i,k} x_i x_k \right) - b_i x_i$$

Partial derivative with respect to  $x_i$  gives the following,

$$\frac{\delta f}{\delta x_i} = Q_{i,i} x_i + \frac{1}{2} \sum_{j \neq i} Q_{j,i} x_j + \frac{1}{2} \sum_{j \neq i} Q_{i,j} x_j - b_i$$

Since  $Q$  is symmetric we can combine the sums to get,

$$\frac{\delta f}{\delta x_i} = \sum_{j=1}^n Q_{i,j} x_j - b_i = (Qx - b)_i \quad (1)$$

Across all rows  $i$  we get that  $\nabla f(x) = Qx - b$ .  $\square$

(b) Show that  $f$  is strictly convex. (Hint. Use facts from section 2.3)

*Proof.* Suppose  $f(x) = \frac{1}{2}x^T Qx - c^T x$  with  $x \in \mathbb{R}^n$ ,  $c \in \mathbb{R}^n$  and  $Q \in \mathbb{R}^{n \times n}$  is a symmetric positive definite matrix. Recall from Section 2.3 that a function is strictly convex if and only if,

$$f''(x) > 0$$

for all  $x \in S$  the feasible set. In the multidimensional case the Hessian matrix must be positive definite. Computing the Hessian of  $f(x)$ , consider equation (1), taking the derivative with respect to a single  $x_j$  in the sum we get,

$$\frac{\delta^2 f}{\delta x_i \delta x_j} = Q_{i,j} \quad (2)$$

Therefore  $\nabla^2 f(x) = Q$  which is positive definite on all  $\mathbb{R}^n$ . Therefore  $f$  is strictly convex.  $\square$

- (c) Suppose  $p$  is a descent direction at  $x$ , so that  $p^T \nabla f(x) < 0$ . Prove that the exact solution of the line search problem  $\min_{\alpha > 0} f(x + \alpha p)$  is,

$$\alpha = \frac{-p^T \nabla f(x)}{p^T Q p}.$$

*Proof.* Let  $g(\alpha) = f(x + \alpha p)$  where  $p$  is a descent direction. Note the minimum of  $g$  is the solution to the line search problem. Substituting  $x + \alpha p$  in  $f(x)$  we get,

$$\begin{aligned} f(x + \alpha p) &= \frac{1}{2}(x + \alpha p)^T Q(x + \alpha p) - c^T(x + \alpha p) \\ &= \frac{1}{2}x^T Qx + \frac{\alpha^2}{2}p^T Qp + \alpha p^T Qx - c^T x - \alpha c^T p \\ &= \left(\frac{1}{2}x^T Qx - c^T x\right) + \frac{\alpha^2}{2}p^T Qp + \alpha p^T(Q(x) - c^T) \\ &= f(x) + \frac{\alpha^2}{2}p^T Qp + \alpha p^T \nabla f(x) \end{aligned}$$

Taking the derivative with respect to  $\alpha$  to compute  $g'(\alpha)$ ,

$$g'(\alpha) = \alpha p^T Qp + p^T \nabla f(x).$$

Setting  $g'(\alpha)$  equal to zero and solving for  $\alpha$ ,

$$\alpha = \frac{-p^T \nabla f(x)}{p^T Qp}.$$

This value of  $\alpha$  must be a minimum since  $g''(\alpha) = p^T Qp > 0$  because  $Q$  is positive definite.  $\square$

**Exercise P14:** (a) Implement steepest descent with optimal step size for quadratic functions (1):

function z = sdquad(x0, Q, c, tol)

As before, stop when  $\|\nabla f(x_k)\| < tol$

*Solution. Code:*

```
function [x, iter] = sdquad(x0, Q, c, tol)

x = x0;
p = -(Q*x - c);
iter = 0;
while norm(p) >= tol
    alpha = -(p'*(Q*x - c))/(p'*Q*p);
    x = x + alpha*p;
    p = -(Q*x - c);
    iter = iter + 1;
end
end
```

□

- (b) Use *sdquad()* to reproduce the result of Example 12.1 on pages 404-405 of the textbook. Specifically, you should get  $k = 216$  iterations using  $tol = 10^{-8}$ .

*Solution. Code:*

```
>> Q = [1 0 0; 0 5 0; 0 0 25]

      1      0      0
      0      5      0
      0      0     25

>> c = [-1 -1 -1]'
>> x0 = [0 0 0]'
>> tol = 10^(-8)
>> [x, iter] = sdquad(x0, Q, c, tol)

x =
    -1.0000
    -0.2000
    -0.0400

iter =
    216
```

□

(c) Now change  $Q$  to

$$Q = \begin{pmatrix} 2.3 & 0.19 & -0.89 \\ 0.19 & 1.84 & 0.32 \\ -0.89 & 0.32 & 1.86 \end{pmatrix}$$

But keep the same  $c$ ,  $x_0$ , and  $tol$ . What is  $x^*$ ? How many iterations does `sdquad()` need? Why is this problem easier than part b?

*Solution.* Running `sdquad()` gives us an  $x^* \approx (-0.7296, -0.3236, -0.8311)^T$  in 26 iterations. Running `eig()` on both  $Q$  matrices we find that our new  $Q$  has relatively closer eigenvalues. One could imagine the effect this has in the two dimensional case where the level curves of our quadric surface objective function look more like circles, so the steepest descent direction actually points towards the minimum. Another idea is to note that in the second problem the identity matrix used in computing the decent direction,

$$p_k = -I^{-1} \nabla f(x_k),$$

is doing a better job at approximating the Hessian,

$$p_k = -(\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

which for quadratic problems as we have shown in the previous problems gives the minimum in one iteration.

**Code:**

```
>> Q1 = [2.3 , 0.19 , -0.89; 0.19 , 1.84 , 0.32; -0.89 , 0.32 , 1.86]
```

```

    2.3000    0.1900   -0.8900
    0.1900    1.8400    0.3200
   -0.8900    0.3200    1.8600
```

```
>> [x1 , iter1] = sdquad(x0 , Q1 , c , tol)
```

```

x1 =
   -0.7296
   -0.3236
   -0.8311
```

```

iter1 =
    26
```

```
>> eig(Q)
```

```

    1
    5
   25
```

>> eig (Q1)

1.0007  
2.0004  
2.9989

□

**Exercise P15:** (a) Let  $x_k$  be a sequence which converges super linearly to  $x_*$ . Show that

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} = 1$$

*Proof.* Suppose  $x_k$  is a sequence which converges super-linearly to  $x_*$ . By definition we know that for  $e_{k+1} = x_{k+1} - x_*$  and  $e_k = x_k - x_*$  we get,

$$\lim_{k \rightarrow \infty} \frac{\|e_{k+1}\|}{\|e_k\|} = \frac{\|x_{k+1} - x_*\|}{\|x_k - x_*\|} = 0.$$

Consider the following limit,

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|}$$

Adding 0 to the numerator in the form of  $x_* - x_*$ , and simplifying we get

$$\begin{aligned} \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} &= \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k + (x_* - x_*)\|}{\|x_k - x_*\|} \\ &= \lim_{k \rightarrow \infty} \frac{\|(x_{k+1} - x_*) + (x_* - x_k)\|}{\|x_k - x_*\|} \\ &\leq \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_*\| + \|x_* - x_k\|}{\|x_k - x_*\|} \\ &= \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_*\| + \|x_k - x_*\|}{\|x_k - x_*\|} \\ &= \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_*\|}{\|x_k - x_*\|} + \frac{\|x_k - x_*\|}{\|x_k - x_*\|} \\ &= \lim_{k \rightarrow \infty} \frac{\|e_{k+1}\|}{\|e_k\|} + 1. \end{aligned}$$

Therefore we get the following,

$$\begin{aligned} \lim_{k \rightarrow \infty} \frac{\|e_{k+1}\|}{\|e_k\|} + 1 &\leq \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} \leq \lim_{k \rightarrow \infty} \frac{\|e_{k+1}\|}{\|e_k\|} + 1 \\ 1 &\leq \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} \leq 1. \end{aligned}$$

So we conclude that,

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} = 1.$$

□

- (b) Explain why a super-linearly converging iterative algorithm for approximating some unknown number  $x_*$  which stops according to the criterion  $\|x_{k+1} - x_k\| < tol$ , for some user-supplied tolerance  $tol > 0$ , is probably going to work acceptably. Does the same apply to iterations which converge only linearly?

*Solution.* From our previous result it's clear that at the limit  $\|x_{k+1} - x_k\| = \|x_k - x_*\|$  so using  $\|x_{k+1} - x_k\|$  as stopping criteria in place of what we actually want, which is  $\|x_k - x_*\|$  is going to work. In a linearly converging algorithm it is not guaranteed that  $\|x_{k+1} - x_k\| = \|x_k - x_*\|$  at the limit. We would likely get something like,

$$-C + 1 \leq \lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x_k\|}{\|x_k - x_*\|} \leq C + 1.$$

□